LLM Scheduler 사용자 가이드

시작하기

이 가이드는 LLM Scheduler 시스템을 설치, 구성, 사용하는 방법을 안내합니다.

---

1. 설치 (Installation)

1.1 사전 요구사항

다음 소프트웨어가 시스템에 설치되어 있어야 합니다.

소프트웨어	최소 버전	권장 버전
Node.js	20.0.0	20 LTS
npm	9.0.0	최신 LTS
Docker	24.0.0	최신 안정 버전
Docker Compose	2.20.0	최신 안정 버전
MongoDB	7.0	7.0+
Redis	7.0	7.2+
Ollama	최신	최신 (선택사항)

1.2 의존성 서비스 시작

Docker Compose 사용 (권장)

```
# 프로젝트 디렉토리로 이동
cd candidates/candidate-1-llm-scheduler

# Redis와 MongoDB 시작
docker-compose up -d

# 서비스 상태 확인
docker-compose ps
```

수동 설치

Redis 설치:

```
# macOS
brew install redis
brew services start redis

# Ubuntu/Debian
sudo apt-get install redis-server
sudo systemctl start redis

# Windows
# Docker Desktop을 사용하거나 WSL2에 설치
```

MongoDB 설치:

```
# macOS
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community

# Ubuntu/Debian
```

```
sudo apt-get install mongodb
sudo systemctl start mongodb

# Windows
# MongoDB Community Server 다운로드 및 설치
```

1.3 애플리케이션 설치

```
# 의존성 설치
npm install

# 환경 변수 설정
cp .env.example .env

# .env 파일 편집
nano .env
```

1.4 Ollama 설치 (선택사항)

로컬 LLM을 사용하려면 Ollama를 설치하세요.

```
# macOS/Linux
curl https://ollama.ai/install.sh | sh

# 모델 다운로드
ollama pull llama2
ollama pull mistral

# 모델 목록 확인
ollama list
```

---

2. 구성 (Configuration)

2.1 환경 변수

.env 파일을 생성하고 다음 변수를 설정합니다.

```
# 서버 설정
NODE_ENV=development
PORT=3000

# Redis 설정
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

# MongoDB 설정
MONGODB_URI=mongodb://localhost:27017/llm-scheduler
MONGODB_DB_NAME=llm-scheduler

# Ollama 설정
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama2

# OpenAI 설정 (선택사항)
OPENAI_API_KEY=your-openai-api-key
OPENAI_MODEL=gpt-3.5-turbo-instruct

# 스케줄러 설정
```

```
DEFAULT_SCHEDULER=fcfs
MAX_CONCURRENT_REQUESTS=5
QUEUE_RETENTION_MS=86400000

# WebSocket 설정
SOCKET_CORS_ORIGIN=http://localhost:3000

# 로깅
LOG_LEVEL=info
```

2.2 스케줄러 알고리즘 선택

.env 파일에서 기본 스케줄러를 설정합니다.

```
# FCFS (기본값)
DEFAULT_SCHEDULER=fcfs

# 우선순위 기반
DEFAULT_SCHEDULER=priority

# MLFQ
DEFAULT_SCHEDULER=mlfq

# WFQ
DEFAULT_SCHEDULER=wfq
```

---

3. 실행 (Running)

3.1 개발 모드

```
# 개발 서버 시작 (핫 리로드 지원)
npm run dev

# 또는 TypeScript 직접 실행
npm run dev:ts
```

서버가 http://localhost:3000에서 시작됩니다.

3.2 프로덕션 모드

```
# TypeScript 컴파일
npm run build

# 프로덕션 서버 시작
npm start

# 또는 PM2 사용 (권장)
npm install -g pm2
pm2 start dist/index.js --name llm-scheduler
pm2 save
pm2 startup
```

3.3 헬스 체크

```
# 시스템 상태 확인
curl http://localhost:3000/api/health

# 응답 예시
{
  "status": "ok",
```

```
  "timestamp": "2025-01-25T10:30:00.000Z",
  "scheduler": {
    "type": "fcfs",
    "status": "running"
  },
  "services": {
    "redis": "connected",
    "mongodb": "connected"
  }
}
```

---

4. 기본 사용법 (Basic Usage)

4.1 첫 번째 요청 보내기

cURL 사용:

```
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "What is quantum computing?",
    "priority": 5,
    "maxTokens": 500,
    "userId": "user-123"
  }'
```

응답:

```
{
  "success": true,
  "requestId": "req-abc123xyz",
  "status": "queued",
  "position": 1,
  "estimatedWaitTime": 5000
}
```

4.2 요청 상태 확인

```
curl http://localhost:3000/api/requests/req-abc123xyz
```

응답:

```
{
  "requestId": "req-abc123xyz",
  "status": "processing",
  "prompt": "What is quantum computing?",
  "result": null,
  "error": null,
  "createdAt": "2025-01-25T10:25:00.000Z",
  "startedAt": "2025-01-25T10:25:05.000Z",
  "completedAt": null
}
```

4.3 대기열 상태 확인

```
curl http://localhost:3000/api/queue/status
```

응답:

```
{
  "scheduler": {
```

```
    "name": "fcfs-queue",
    "type": "fcfs",
    "concurrency": 2
  },
  "queue": {
    "waiting": 3,
    "active": 2,
    "completed": 15,
    "failed": 0,
    "delayed": 0,
    "paused": false
  }
}
```

---

5. 스케줄러 사용 (Scheduler Usage)

5.1 FCFS (First-Come, First-Served)

가장 간단한 스케줄러로, 요청 순서대로 처리합니다.

```
# FCFS로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "fcfs"}'
```

사용 케이스: - 간단한 배치 작업 - 공평한 처리가 필요한 경우 - 우선순위가 중요하지 않은 경우

5.2 Priority Queue

우선순위가 높은 요청을 먼저 처리합니다.

```
# 우선순위로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "priority"}'

# 높은 우선순위 요청
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Urgent request",
    "priority": 10,
    "userId": "premium-user"
  }'
```

사용 케이스: - 유료 사용자 우선 처리 - 긴급한 요청 처리 - SLA 보장이 필요한 경우

5.3 MLFQ (Multi-Level Feedback Queue)

빠른 응답과 공정성의 균형을 제공합니다.

```
# MLFQ로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{
    "algorithm": "mlfq",
    "config": {
      "queues": 3,
      "timeQuantum": 5000
```

```
    }
  }'
```

사용 케이스: - 혼합된 작업 부하 - 응답 시간과 공정성이 모두 중요한 경우 - 짧은 요청을 빠르게 처리하려는 경우

5.4 WFQ (Weighted Fair Queuing)

테넌트별로 공정한 자원 분배를 제공합니다.

```
# WFQ로 변경
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{
    "algorithm": "wfq",
    "config": {
      "tenants": {
        "tenant-a": 50,
        "tenant-b": 30,
        "tenant-c": 20
      }
    }
  }'

# 테넌트별 요청
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Tenant A request",
    "tenantId": "tenant-a",
    "priority": 5
  }'
```

사용 케이스: - 멀티테넌트 SaaS - 공정한 자원 분배 필요 - QoS 보장이 필요한 경우

---

6. 실시간 모니터링 (Real-time Monitoring)

6.1 WebSocket 연결

```
const io = require('socket.io-client');

const socket = io('http://localhost:3000');

// 연결 성공
socket.on('connected', (data) => {
  console.log('Connected:', data);
});

// 대기열 업데이트 구독
socket.emit('subscribe:queue');

// 대기열 업데이트 수신
socket.on('queue:updated', (data) => {
  console.log('Queue updated:', data);
  // {
  //   waiting: 5,
  //   active: 2,
  //   completed: 20
```

```
  // }
});
```

6.2 대시보드 사용

```
# 대시보드 시작 (별도 프로세스)
npm run dashboard

# 브라우저에서 접속
open http://localhost:3001
```

대시보드에서 다음 정보를 확인할 수 있습니다: - 실시간 대기열 상태 - 요청 처리 메트릭 - 알고리즘 성능 비교 - 시스템 리소스 사용량

---

7. 클라이언트 라이브러리 (Client Libraries)

7.1 Node.js 클라이언트

```
const { LLMSchedulerClient } = require('@llm-scheduler/client');

const client = new LLMSchedulerClient({
  baseURL: 'http://localhost:3000/api',
  apiKey: 'your-api-key' // 선택사항
});

async function main() {
  try {
    // 요청 제출
    const { requestId } = await client.submit({
      prompt: 'Explain machine learning',
      priority: 7,
      maxTokens: 1000
    });

    console.log('Request ID:', requestId);

    // 결과 대기
    const result = await client.waitForResult(requestId, {
      timeout: 60000,
      pollInterval: 1000
    });

    console.log('Result:', result);

  } catch (error) {
    console.error('Error:', error);
  }
}

main();
```

7.2 Python 클라이언트

```
from llm_scheduler import LLMSchedulerClient
import asyncio

async def main():
    client = LLMSchedulerClient(
```

```python
        base_url="http://localhost:3000/api",
        api_key="your-api-key"  # 선택사항
    )

    try:
        # 요청 제출
        request_id = await client.submit(
            prompt="Explain machine learning",
            priority=7,
            max_tokens=1000
        )

        print(f"Request ID: {request_id}")

        # 결과 대기
        result = await client.wait_for_result(
            request_id,
            timeout=60,
            poll_interval=1
        )

        print(f"Result: {result}")

    except Exception as e:
        print(f"Error: {e}")

asyncio.run(main())
```

---

8. 일반적인 사용 시나리오 (Common Use Cases)

8.1 배치 처리

여러 요청을 한 번에 제출하고 모두 완료될 때까지 기다립니다.

```javascript
const prompts = [
  'What is AI?',
  'Explain machine learning',
  'What is deep learning?'
];

const requestIds = await Promise.all(
  prompts.map(prompt =>
    client.submit({ prompt, priority: 5 })
  )
);

const results = await Promise.all(
  requestIds.map(id =>
    client.waitForResult(id)
  )
);

console.log('All results:', results);
```

8.2 스트리밍 응답

긴 응답을 실시간으로 수신합니다.

```
const socket = io('http://localhost:3000');

socket.emit('subscribe:request', { requestId: 'req-123' });

socket.on('request:stream', (data) => {
  console.log('Stream chunk:', data.chunk);
});

socket.on('request:completed', (data) => {
  console.log('Complete:', data.result);
});
```

8.3 우선순위 조정

실행 중인 요청의 우선순위를 변경합니다.

```
# 우선순위 상향
curl -X PATCH http://localhost:3000/api/requests/req-123/priority \
  -H "Content-Type: application/json" \
  -d '{"priority": 10}'
```

---

9. 문제 해결 (Troubleshooting)

9.1 일반적인 문제

문제: 서버가 시작되지 않음

해결:

```
# 포트 충돌 확인
lsof -i :3000

# 다른 포트 사용
PORT=3001 npm run dev

# 또는 충돌하는 프로세스 종료
kill -9 <PID>
```

문제: Redis 연결 실패

해결:

```
# Redis 상태 확인
redis-cli ping

# Redis 시작
brew services start redis  # macOS
sudo systemctl start redis  # Linux

# 또는 Docker 사용
docker-compose up -d redis
```

문제: MongoDB 연결 실패

해결:

```
# MongoDB 상태 확인
mongosh --eval "db.adminCommand('ping')"

# MongoDB 시작
```

```
brew services start mongodb-community  # macOS
sudo systemctl start mongodb  # Linux

# 또는 Docker 사용
docker-compose up -d mongodb
```

문제: Ollama 연결 실패

해결:

```
# Ollama 상태 확인
curl http://localhost:11434/api/tags

# Ollama 시작
ollama serve

# 모델 확인
ollama list
ollama pull llama2
```

9.2 로그 확인

```
# 개발 모드 로그
npm run dev

# 프로덕션 모드 로그
pm2 logs llm-scheduler

# 로그 레벨 조정
LOG_LEVEL=debug npm run dev
```

9.3 진단 도구

```
# 시스템 헬스 체크
curl http://localhost:3000/api/health

# 대기열 상태 확인
curl http://localhost:3000/api/queue/status

# 메트릭 확인
curl http://localhost:3000/api/metrics

# Redis 모니터링
redis-cli monitor

# MongoDB 모니터링
mongosh
> use llm-scheduler
> db.requests.find().pretty()
```

---

10. 성능 튜닝 (Performance Tuning)

10.1 동시 처리량 조정

```
# .env 파일
MAX_CONCURRENT_REQUESTS=10

# 또는 코드에서 설정
const scheduler = new FCFSScheduler({
```

```
  concurrency: 10,
  name: 'fcfs-queue'
});
```

10.2 메모리 최적화

```
# .env 파일
QUEUE_RETENTION_MS=3600000  # 1시간

# Redis 최대 메모리 설정
redis-cli CONFIG SET maxmemory 1gb
redis-cli CONFIG SET maxmemory-policy allkeys-lru
```

10.3 캐싱 활성화

```
# .env 파일
ENABLE_CACHE=true
CACHE_TTL=3600
```

---

11. 모범 사례 (Best Practices)

11.1 요청 최적화

```
// 좋은 예시: 적절한 토큰 수
{
  prompt: "Brief explanation",
  maxTokens: 500  // 필요한 만큼만
}

// 나쁜 예시: 과도한 토큰
{
  prompt: "Brief explanation",
  maxTokens: 4000  // 낭비
}
```

11.2 우선순위 사용

```
// 긴급한 요청
client.submit({ prompt, priority: 10 });

// 일반 요청
client.submit({ prompt, priority: 5 });

// 배치 작업
client.submit({ prompt, priority: 1 });
```

11.3 에러 처리

```
try {
  const result = await client.waitForResult(requestId);
} catch (error) {
  if (error.code === 'TIMEOUT') {
    // 재시도 로직
  } else if (error.code === 'RATE_LIMIT') {
    // 백오프
    await new Promise(resolve => setTimeout(resolve, 1000));
  } else {
    // 기타 에러 처리
  }
}
```

---

12. 다음 단계 (Next Steps)

- API 문서 - 전체 API 레퍼런스
- 아키텍처 문서 - 시스템 설계 이해
- GitHub 저장소 - 소스 코드
- 이슈 트래커 - 버그 보고

---

13. 지원 및 피드백

- 이메일: support@llm-scheduler.dev
- **Discord**: 커뮤니티 서버
- **GitHub Issues**: 버그 보고

문서 버전: 1.0.0 최종 업데이트: 2025-01-25 유지보수 담당자: LLM Scheduler 팀